Docker Commands and Details

A quick command reference followed by a detailed explanation of how the most important commands work.

Part 1: Command reference

Images

```
docker pull nginx:latest          # download an image
docker images                     # list local images
docker build -t myapp:1.0 .       # build from a Dockerfile
docker tag myapp:1.0 user/myapp:1.0
docker push user/myapp:1.0        # upload to a registry
docker rmi myapp:1.0              # remove an image
docker history myapp              # show image layers
docker save -o app.tar myapp      # export image to a file
docker load -i app.tar            # import image from a file
```

Running containers

```
docker run nginx                          # run in foreground
docker run -d nginx                       # detached (background)
docker run -d -p 8080:80 nginx            # map host:container port
docker run -d --name web nginx            # give it a name
docker run -it ubuntu bash                # interactive shell
docker run --rm alpine echo hi            # auto-remove on exit
docker run -e KEY=value nginx             # environment variable
docker run -v mydata:/data nginx          # mount a volume
docker run --memory=512m --cpus=1 nginx   # resource limits
docker run --restart unless-stopped nginx
```

Managing containers

```
docker ps                 # running containers
docker ps -a              # all containers
docker stop web           # graceful stop
docker start web
docker restart web
docker kill web           # force stop
docker rm web             # remove (add -f to force)
docker pause web / unpause web
```

Inspecting and debugging

```
docker logs web                 # view logs
docker logs -f --tail 100 web   # follow last 100 lines
docker exec -it web sh          # shell into a running container
docker inspect web              # detailed JSON info
docker stats                    # live CPU/memory usage
docker top web                  # processes inside container
docker cp web:/app/file.txt .   # copy files out
docker diff web                 # filesystem changes
```

Volumes

```
docker volume create mydata
docker volume ls
docker volume inspect mydata
docker volume rm mydata
```

Networks

```
docker network create mynet
docker network ls
docker network inspect mynet
docker network connect mynet web
docker network disconnect mynet web
docker run -d --network mynet nginx
```

Docker Compose

```
docker compose up -d       # start all services
docker compose down        # stop and remove
docker compose down -v     # also remove volumes
docker compose ps
docker compose logs -f
docker compose build
docker compose exec web sh
docker compose restart
```

Cleanup

```
docker system df           # disk usage
docker container prune     # remove stopped containers
docker image prune -a      # remove unused images
docker volume prune
docker system prune -a     # remove everything unused
```

System and registry

```
docker version
docker info
docker login
docker logout
docker search nginx
```

Part 2: Docker commands in detail

docker run

`docker run` is really three steps in one: pull the image if it is missing, create a container, and start it.

```
docker run -d -p 8080:80 --name web -e MODE=prod -v mydata:/data --restart unless-stopped nginx
```

Flag	What it does
`-d`	Runs in the background and prints the container ID. Without it, your terminal is attached to the container output.
`-p 8080:80`	Maps host port 8080 to container port 80. Order is always host:container.
`--name web`	Names the container. Otherwise Docker generates a random one like quirky_turing.
`-e MODE=prod`	Sets an environment variable inside the container.
`-v mydata:/data`	Mounts the named volume mydata at /data. Data survives container removal.
`--restart unless-stopped`	Restarts on crash or reboot unless you stopped it manually. Other options: no, on-failure, always.
`-it`	-i keeps stdin open and -t allocates a terminal. Together they give you an interactive shell.
`--rm`	Deletes the container automatically when it exits. Good for one-off tasks.

Flag	What it does
`--memory, --cpus`	Caps resources via cgroups. Exceeding the memory limit gets the process killed (OOM).

stop vs kill vs rm

- `docker stop` sends `SIGTERM`, waits 10 seconds by default, then sends `SIGKILL`. This lets your app shut down cleanly (close connections, flush data).
- `docker kill` sends `SIGKILL` immediately, with no cleanup.
- `docker rm` deletes a stopped container. A stopped container still exists, along with its filesystem and logs, until you remove it. `rm -f` stops and removes in one go.

ps, logs, exec, inspect

- `docker ps` shows only running containers. `-a` includes exited ones, which is how you find a container that crashed.
- `docker logs` shows whatever the main process wrote to stdout/stderr. `-f` follows live and `--tail 100` limits output.
- `docker exec -it web sh` starts a new process inside an already-running container. Exiting that shell does not stop the container. Use `bash` instead of `sh` if the image has it.
- `docker run -it ubuntu bash` is different: it creates a new container whose main process is bash.
- `docker inspect` returns full JSON (IP address, mounts, env vars, config). Filter with `docker inspect -f '{{.NetworkSettings.IPAddress}}' web`.

Building images

```
docker build -t myapp:1.0 .
```

- `-t myapp:1.0` sets name and tag. Without a tag, it defaults to `latest`.
- The `.` is the **build context**: the folder sent to Docker. `COPY` can only access files inside it, so use a `.dockerignore` to exclude things like `node_modules` and `.git`.
- Docker runs each Dockerfile instruction in order and **caches each layer**. If a line and everything before it is unchanged, the cached layer is reused. That is why you copy `package.json` and install dependencies before copying the rest of your code: code changes often, dependencies do not.

Tagging and pushing: a tag is just a label pointing at an image. `docker tag myapp:1.0 user/myapp:1.0` adds a second name (no copy is made), and the `user/` prefix tells `docker push` which registry account to use.

Volumes vs bind mounts

```
docker run -v mydata:/data nginx          # named volume (managed by Docker)
docker run -v $(pwd)/src:/app/src nginx   # bind mount (a host folder)
```

- **Named volumes** are stored and managed by Docker. They are best for databases and persistent app data.
- **Bind mounts** link a specific host path into the container. They are best for development, since edits on your machine appear instantly inside the container.

Without either, anything written inside a container is lost when the container is removed.

Networks

On a **user-defined network**, containers can reach each other **by container name**:

```
docker network create mynet
docker run -d --name db --network mynet postgres
docker run -d --name api --network mynet myapp   # can connect to "db:5432"
```

The default `bridge` network does not provide this name-based DNS, which is why custom networks are recommended. Note that `-p` is only needed to reach a container from your **host**. Containers on the same network can talk to each other on any port without publishing it.

Docker Compose

Compose describes multi-container apps in a `compose.yaml` file instead of long `docker run` commands:

```
services:
  web:
    build: .
    ports: ["3000:3000"]
    depends_on: [db]
  db:
    image: postgres:16
    volumes: [pgdata:/var/lib/postgresql/data]
volumes:
  pgdata:
```

- `docker compose up -d` creates the network, volumes, and containers, and starts everything.
- `docker compose down` removes containers and networks but **keeps volumes**. Add `-v` to delete volumes too (this destroys data).

Cleanup

These commands delete things, so know what each targets:

- `docker container prune` removes all stopped containers.
- `docker image prune` removes dangling (untagged) images. With `-a`, it removes all images not used by a container.
- `docker volume prune` removes volumes not attached to any container. Be careful, as this can delete database data.
- `docker system prune -a` does the broad sweep. Check `docker system df` first to see what is using space.

Common mistakes

- Forgetting `-p` and then wondering why `localhost` does not work.
- Expecting data to persist without a volume.
- Using `latest` tags in production, which makes builds non-reproducible.
- Running as root inside containers when it is not needed.
- Putting `COPY . .` before dependency installation, which breaks caching.